系统分析第三次作业

系统分析第三次作业
- 题目描述
- RBAC权限控制机制分析
- 应用实现
- 完整代码如下
  - User
  - Role
  - Document
  - Permission
  - UserInfoRetriever
  - UserManager
  - PermissionChecker
  - PermissionManagement
  - DocumentManager
  - DatabaseConnectionManager
  - LoginFrame
  - NotebookApp

题目描述

请基于权限的用户访问控制（RBAC）机制的原理，并在基于监听器模式和事件响应机制下，在事件处理的机制上实现一个RBAC的应用实例。其中要求：

1）完整的代码实现；

2）页面操作的显示，与角色和权限控制的实现

3）对RBAC权限控制机制进行分析（例如对页面、对象、功能按钮之间控制的异同）

扩展：RBAC是一个模型簇，可以展开对比分析。

RBAC权限控制机制分析

RBAC即Role-Based Access Control，中文意思是：基于角色（Role）的访问控制。这是一种广泛应用于计算机系统和网络安全领域的访问控制模型。

简单来说，就是通过将权限分配给角色，再将角色分配给用户，来实现对系统资源的访问控制。一个用户拥有若干角色，每一个角色拥有若干权限。这样，就构造成“用户-角色-权限”的授权模型。在这种模型中，用户与角色之间，角色与权限之间，一般者是多对多的关系。

与传统的将用户与权限直接关联不同，RBAC的本质是单纯地用户和权限进行解耦，将用户与角色、角色与权限关联。这样方便对拥有相同特征的多个用户的权限进行管理，也符合系统低耦合度的要求。

具体而言，RBAC模型定义了以下几个核心概念：

角色（Role）：角色是指在系统中具有一组相关权限的抽象概念，代表了用户在特定上下文中的身份或职能，例如管理员、普通用户等。

权限（Permission）：权限是指对系统资源进行操作的许可，如读取、写入、修改等。权限可以被分配给角色。

用户（User）：用户是指系统的实际使用者，每个用户可以被分配一个或多个角色。

分配（Assignment）：分配是指将角色与用户关联起来，以赋予用户相应的权限。

根据控制对象的不同，控制粒度也会不同，在一般的开发中，主要有对页面、菜单和控件的控制。

页面控制：每个用户登录以后根据其分配到的角色权限跳转到不同的主页面，此时软件的整个页面是与角色相联系的，若有n个角色，就会对应n个主页面。显然，这种控制粒度是较粗的，即便不同的角色有一些共同的页面元素也需要创建一个新的页面，这会造成大量代码的复制粘贴，十分不推荐使用。

菜单控制：每个用户登录以后可以跳转到相同的主页面，这时候可以根据其拥有的菜单控制权限显示菜单，没有权限的菜单对用户是不可见的，从而呈现不同的页面效果。此时的控制粒度居中，可以提高对代码的重用率。

控件控制：这里的控制对象可以是页面中的控件，如按钮，输入框等，可以与监听事件机制相结合，实现对控件操作的权限控制。如判断用户是否可以点击相应的功能按钮、是否可以对输入框进行输入等。此时实现了细粒度的权限控制，使系统安全性更高。

RBAC（Role-Based Access Control）模型簇是一组基于角色的访问控制模型的集合，它为不同的应用场景和需求提供了灵活多样的权限管理解决方案。

一、RBAC0 基本模型

RBAC0 是 RBAC 模型簇的基础模型，它定义了用户、角色和权限之间的基本关系。在这个模型中，用户被分配到一个或多个角色，每个角色被赋予一组特定的权限。用户通过其所属的角色来获得相应的权限，从而能够访问系统中的资源和执行特定的操作。

二、RBAC1 角色层次模型

RBAC1 在 RBAC0 的基础上引入了角色层次的概念。角色层次是一种角色之间的继承关系，高级角色可以继承低级角色的权限。这样可以减少权限分配的工作量，提高权限管理的效率。例如，一个经理角色可以继承普通员工角色的权限，同时还拥有一些额外的管理权限。

三、RBAC2 约束模型

RBAC2 在 RBAC0 的基础上增加了各种约束条件，以进一步增强权限管理的安全性和灵活性。这些约束条件可以包括：

1. 互斥角色约束：某些角色不能同时分配给同一个用户，以防止权限冲突和滥用。
2. 基数约束：限制一个用户可以拥有的角色数量或一个角色可以分配的用户数量。
3. 先决条件约束：用户必须先拥有某个角色才能被分配到另一个角色。

四、RBAC3 综合模型

RBAC3 是 RBAC1 和 RBAC2 的综合，它同时包含了角色层次和约束条件。这个模型既提供了角色之间的继承关系，又能够通过约束条件来确保权限管理的安全性和合理性。

五、应用场景

RBAC 模型簇在各种领域都有广泛的应用，例如：

1. 企业管理系统：可以根据不同的岗位和职责为员工分配相应的角色，实现精细化的权限管理。
2. 电子商务平台：区分买家、卖家、管理员等不同角色，保障交易的安全和平台的正常运行。
3. 医疗信息系统：确保只有授权的医疗人员能够访问患者的病历和医疗数据，保护患者的隐私。

应用实现

下是我将利用RBAC模型，结合事件监听机制，设计的一个note系统主要角色包括：

- 普通用户：可以登录系统，进行笔记的编写，以及管理自己的的笔记，包括重新编辑，删除。
- 管理员：管理员主要负责用户的相关操作，包括新增用户，管理已有用户，包括重新编辑（重命名，修改密码，修改角色），删除。
- 权限管理员：管理已有角色所具有的权限

系统数据库设计用户表（users）

字段	字段类型	描述
id	INT	自增主键，用户唯一标识
username	VARCHAR(255)	用户名
password	VARCHAR(255)	用户密码

角色表（roles）

字段	字段类型	描述
id	INT	自增主键，角色唯一标识
role_name	VARCHAR(255)	角色名称

权限表（permissions）

字段	字段类型	描述
id	INT	自增主键，权限唯一标识
permission_name	VARCHAR(255)	权限名称

用户角色关联表（user_roles）

字段	字段类型	描述
user_id	INT	外键，关联 users 表的 id
role_id	INT	外键，关联 roles 表的 id

角色权限关联表（role_permissions）

字段	字段类型	描述
role_id	INT	外键，关联 roles 表的 id
permission_id	INT	外键，关联 permissions 表的 id

文档表（documents）

字段	字段类型	描述
id	INT	自增主键，文档唯一标识
title	VARCHAR(255)	文档标题
content	TEXT	文档内容
user_id	INT	外键，关联 users 表的 id

一、用户管理

用户表（users）：存储系统中的用户信息，包括用户名和密码。每个用户都有一个唯一的 ID 作为标识。

二、角色管理

1. **角色表（roles）**：定义了不同的角色名称，普通用户、管理员、权限管理员。每个角色都有一个唯一的 ID。
2. 系统管理员可以根据业务需求创建新的角色，并为不同的角色分配不同的权限。

三、权限管理

1. **权限表（permissions）**：记录各种具体的权限名称，如创建文档、编辑文档、删除文档、查看文档列表、添加用户、编辑用户、删除用户、管理权限。每个权限都有一个唯一的 ID。
2. 系统管理员可以将其分配给不同的角色。

四、用户角色关联

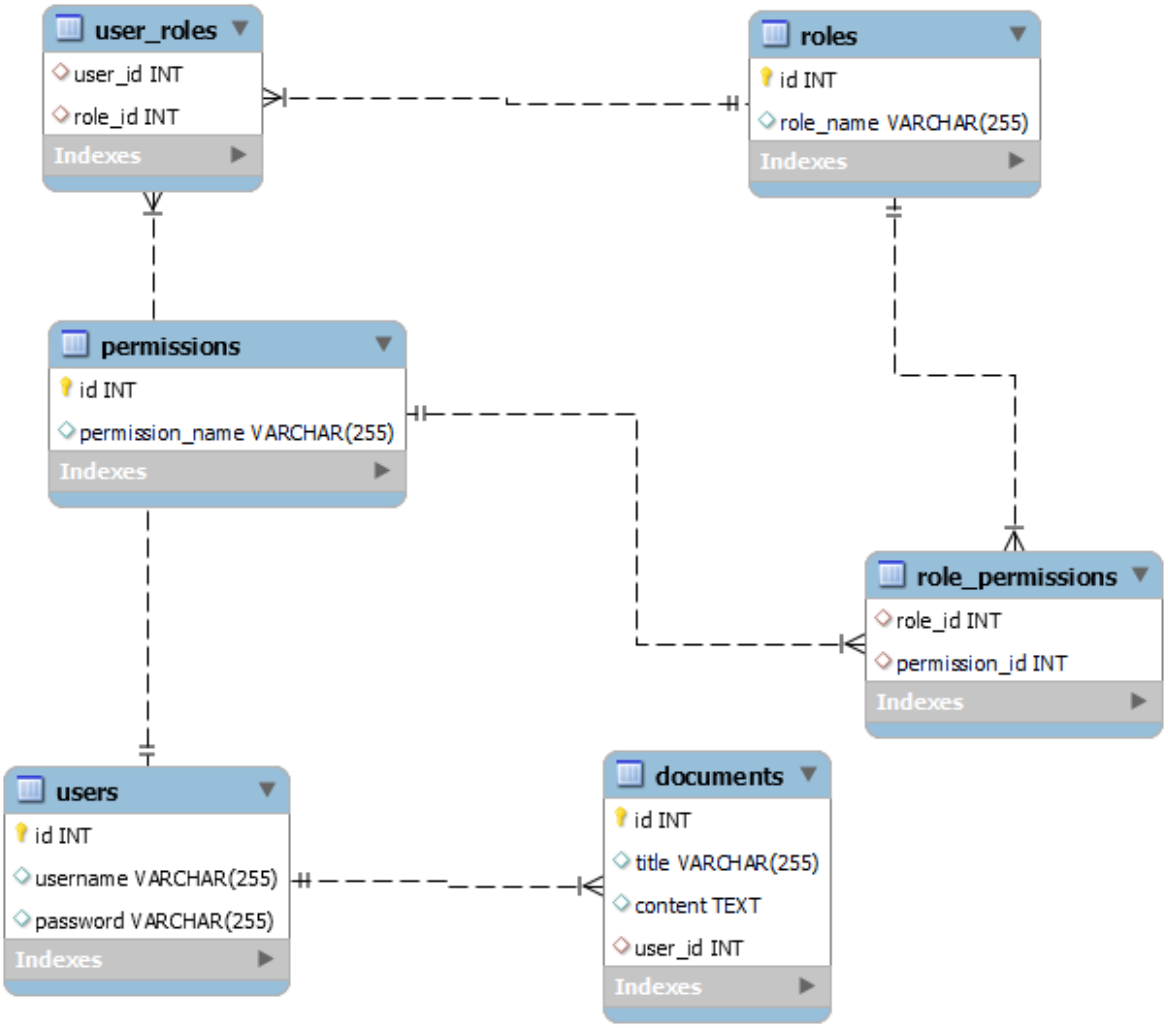
1. **用户角色关联表（user_roles）**：建立用户和角色之间的多对多关系。一个用户可以拥有多个角色，一个角色也可以被多个用户拥有。
2. 当用户注册或被创建时，可以为其分配一个或多个角色。这样，用户就可以通过其拥有的角色来获得相应的权限。

五、角色权限关联

1. **角色权限关联表（role_permissions）**：建立角色和权限之间的多对多关系。一个角色可以拥有多个权限，一个权限也可以被多个角色拥有。
2. 系统管理员可以根据不同的角色需求，为角色分配相应的权限。例如，普通用户角色可能被分配创建、编辑、删除和查看文档列表的权限；管理员角色可能拥有更多的权限，如添加、编辑和删除用户等。

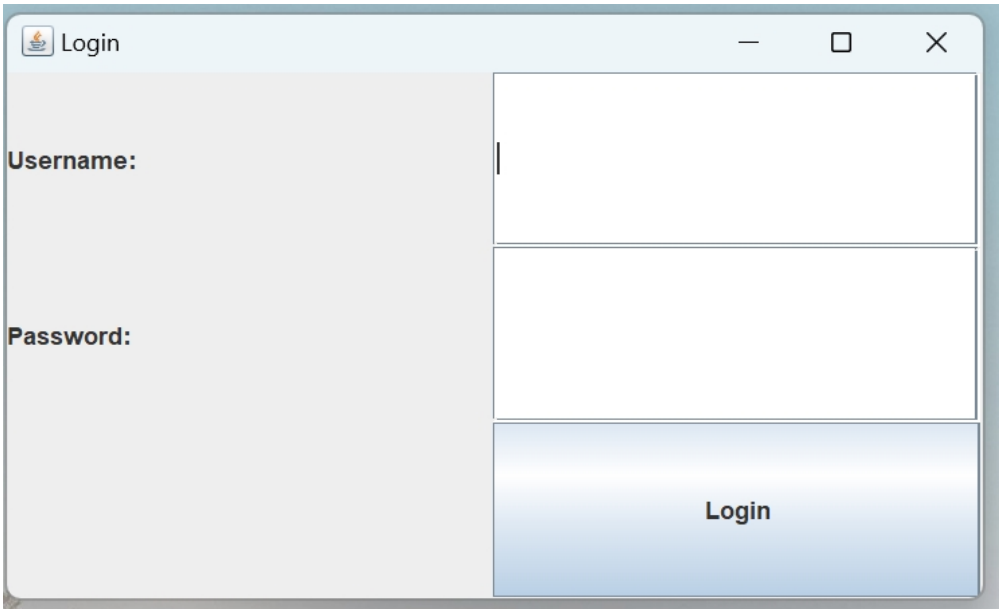
六、文档管理

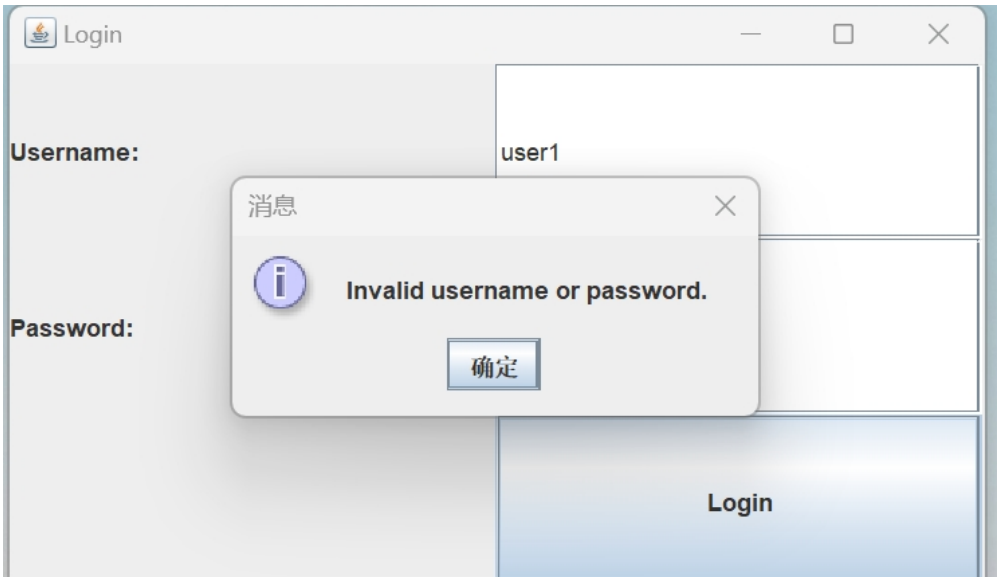
1. **文档表（documents）**：存储用户创建的文档信息，包括文档标题、内容和创建用户的 ID。
2. 用户可以创建、编辑、删除自己创建的文档。



登录界面:

用户输入用户名和密码进行登录，错误则弹窗提醒，正确则进入系统



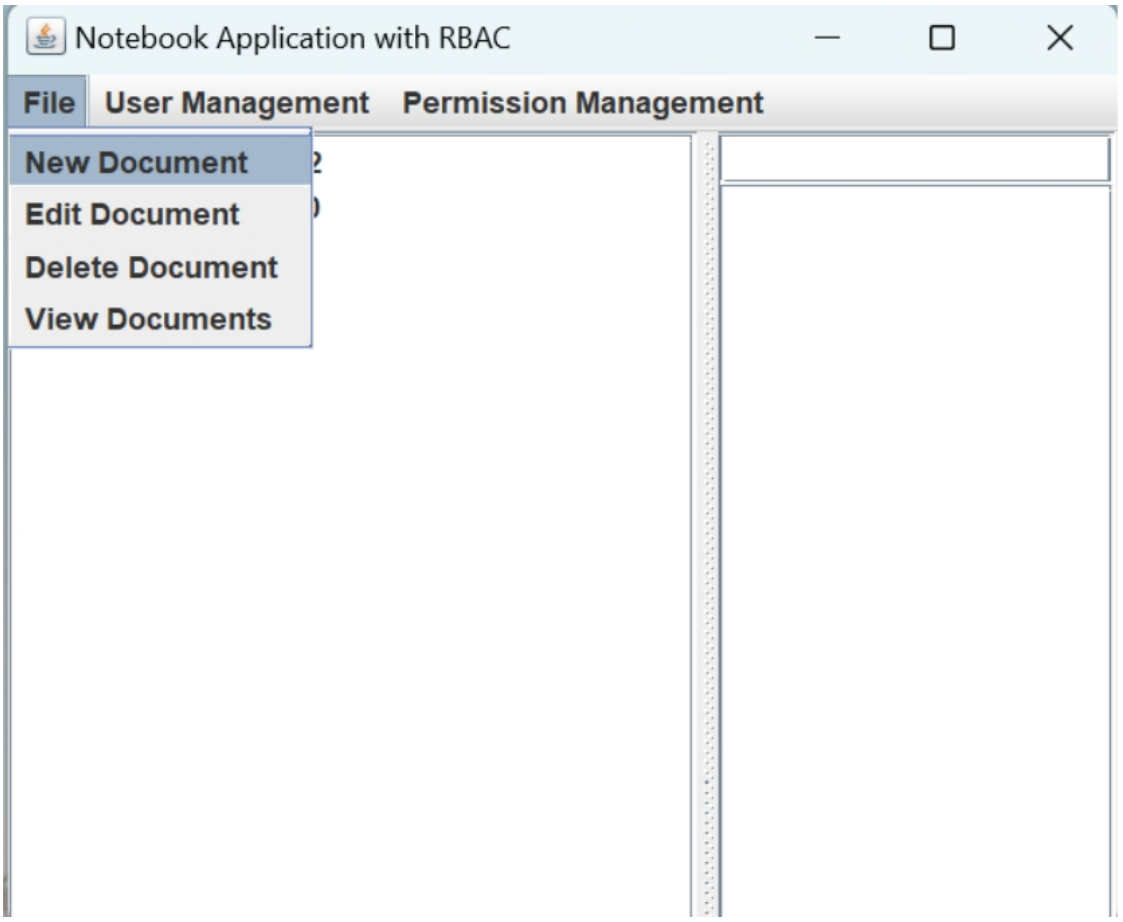


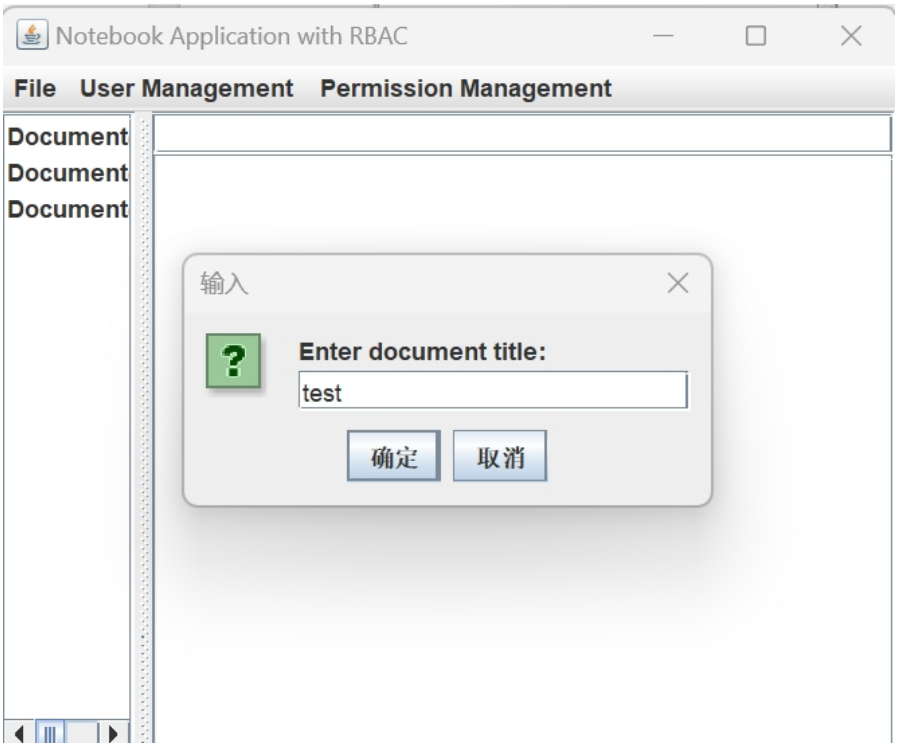
登录事件代码

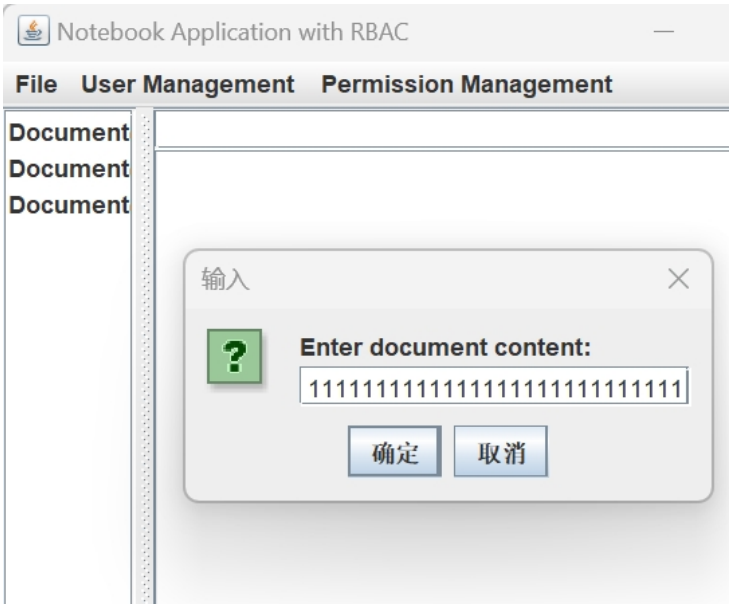
```
loginButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String username = usernameField.getText();
        char[] passwordChars = passwordField.getPassword();
        String password = new String(passwordChars);
        User user = UserInfoRetriever.getUser(username);
        if (user!= null && user.getPassword().equals(password)) {
            dispose();
            new NotebookAppWithRBAC(user);
        } else {
            JOptionPane.showMessageDialog(LoginFrame.this, "Invalid username or
password.");
        }
    }
});
```

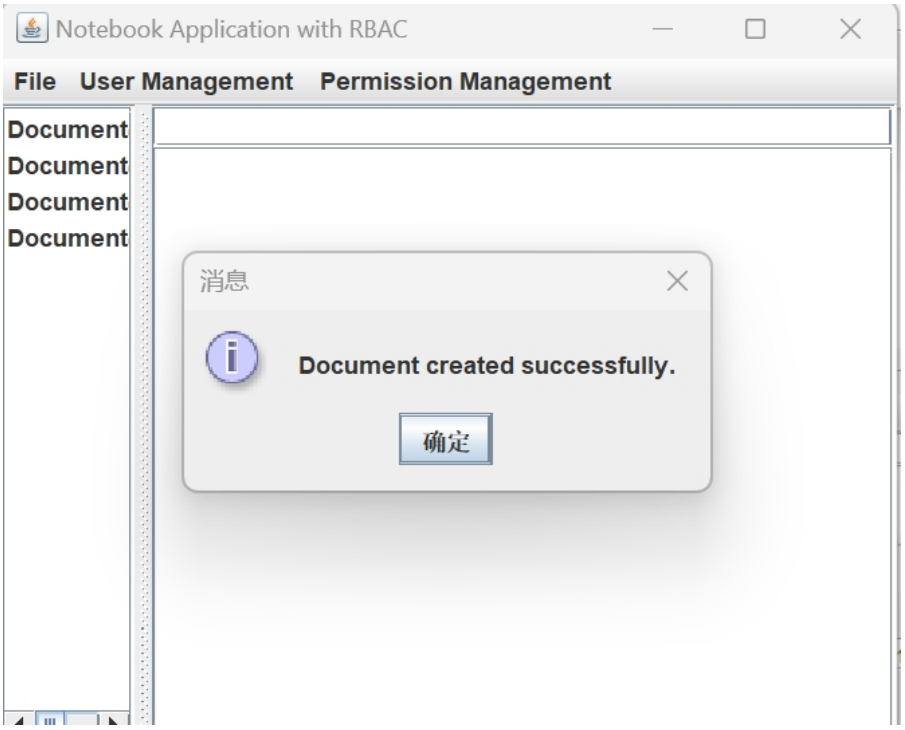
普通用户：

可以创建新文档

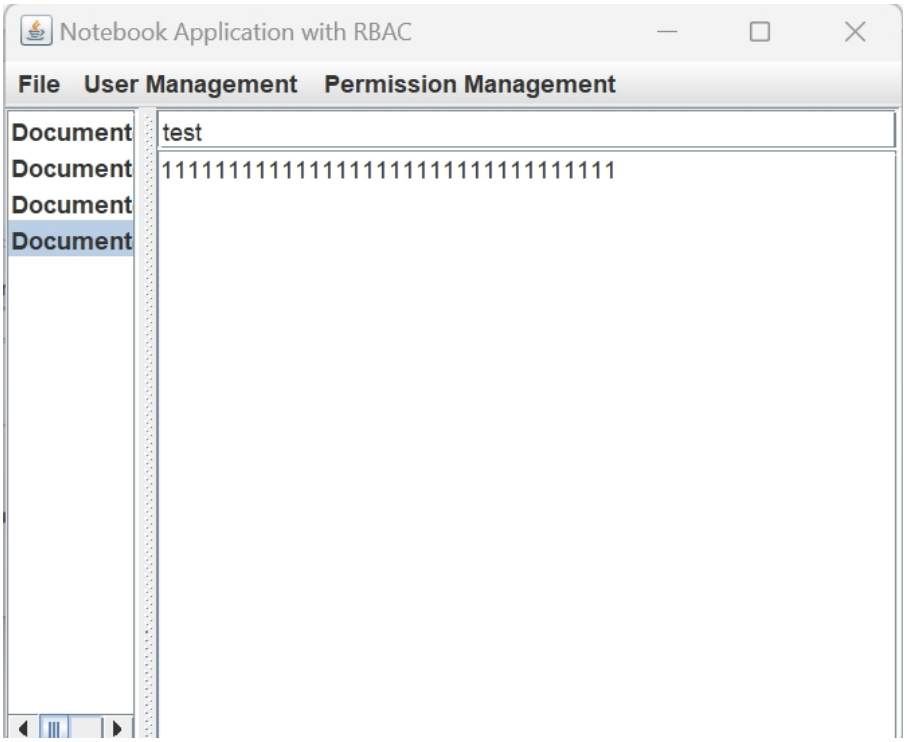

Notebook Application with RBAC
File
User Management
Permission Management
New Document
Edit Document
Delete Document
View Documents


Notebook Application with RBAC
File
User Management
Permission Management
Document
Document
Document
输入
Enter document title:
test
确定
取消


Notebook Application with RBAC
File
User Management
Permission Management
Document
Document
Document
输入
Enter document content:
11111111111111111111111111111
确定
取消


Notebook Application with RBAC
File
User Management
Permission Management
Document
Document
Document
Document
消息
Document created successfully.
确定

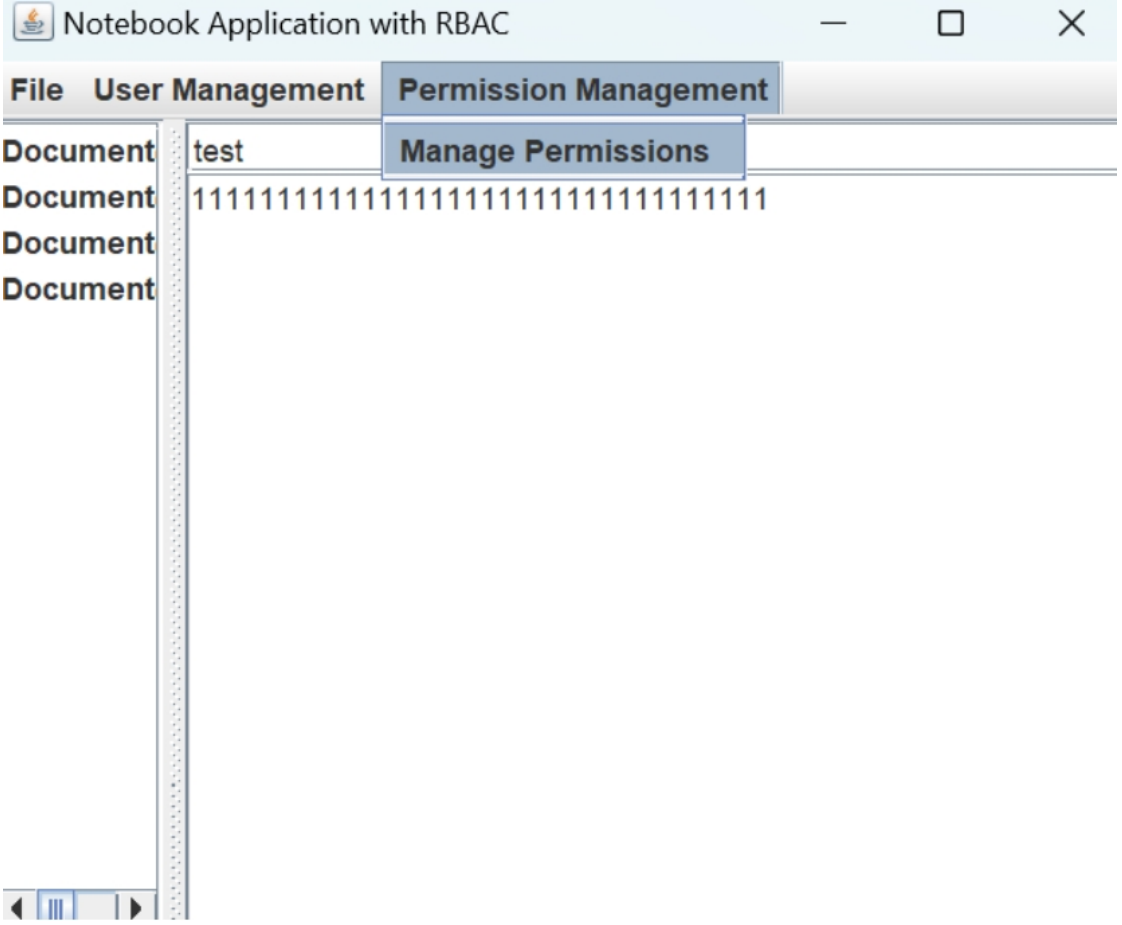
创建成功

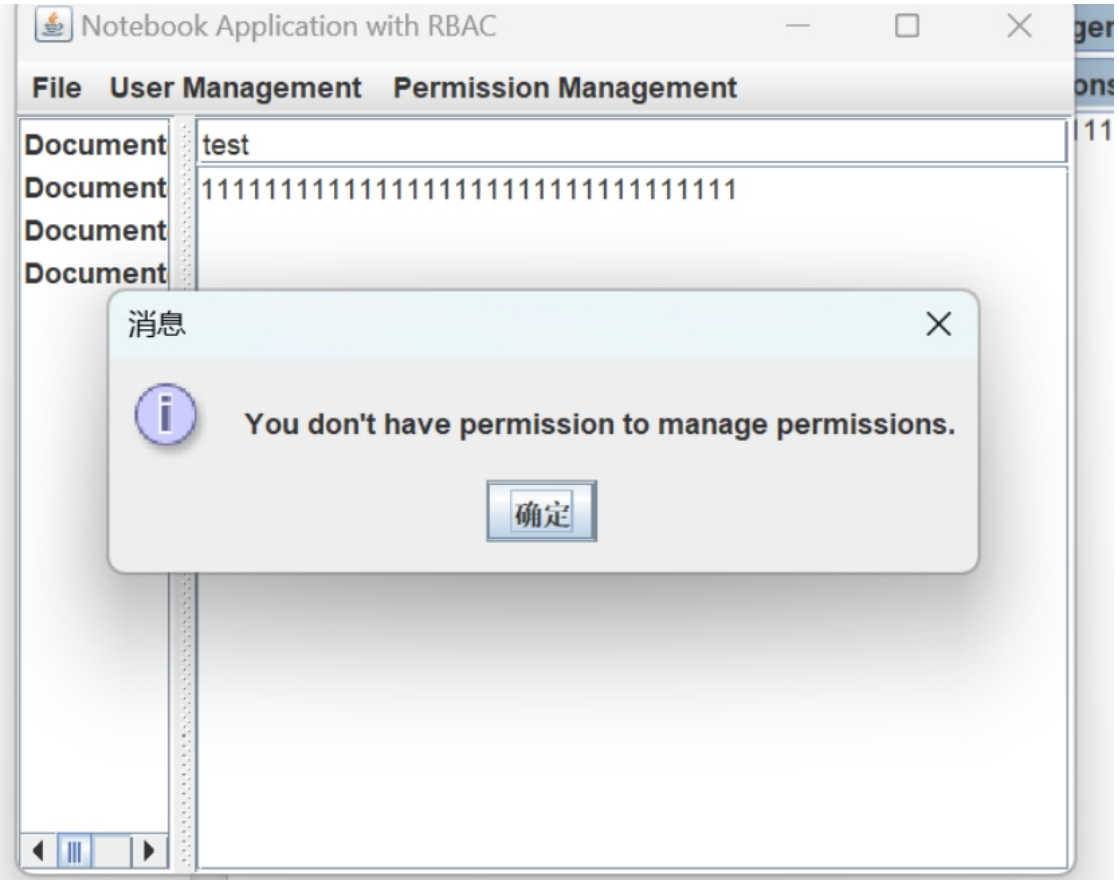


相关事件

```
newDocumentItem.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (currentUser!= null) {
            boolean hasPermission = PermissionChecker.checkPermission(currentUser, ne
            if (hasPermission) {
                String title = JOptionPane.showInputDialog(frame, message: "Enter docu
```

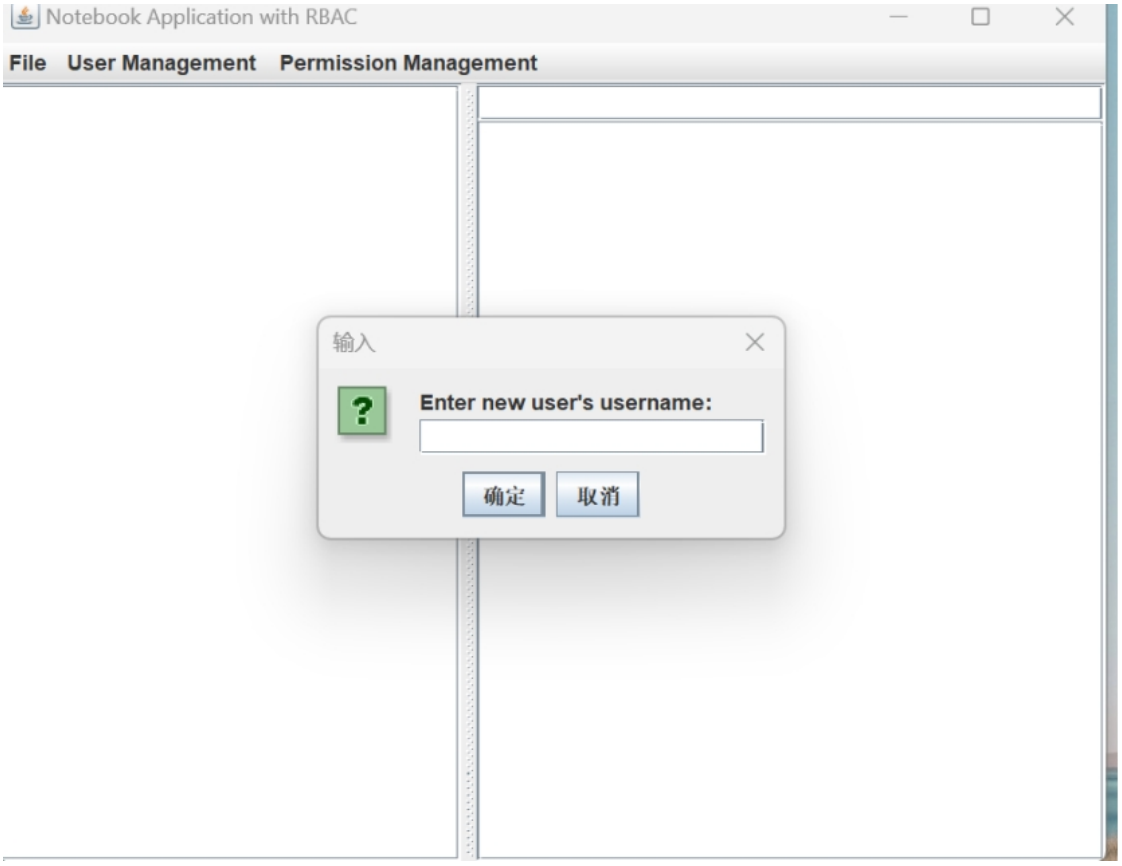
当普通用户尝试进行权限管理，由于没有相应权限，无法操作





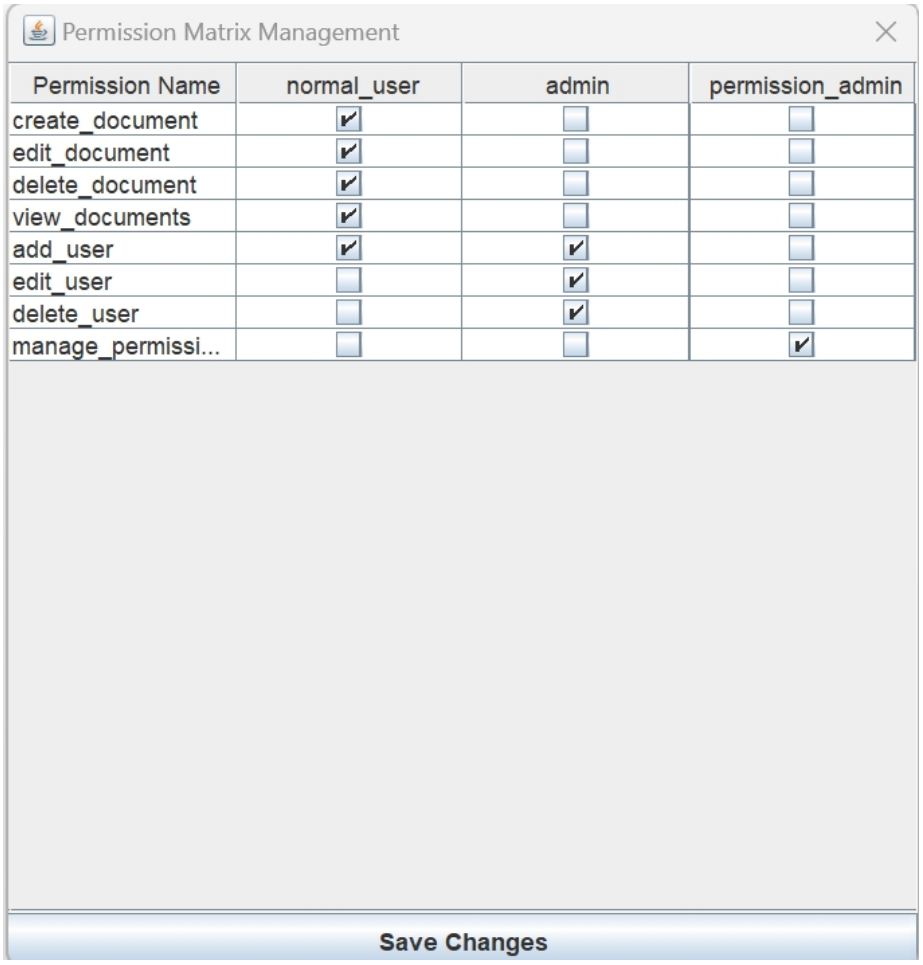

```
    managePermissionsItem.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            if (currentUser!= null && PermissionChecker.checkPermission(currentUser,
```

管理员可以进行用户的管理



```
addUserItem.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (currentUser!= null && PermissionChecker.checkPermission(currentUser, new
            String username = JOptionPane.showInputDialog(frame, message: "Enter new
            if (username!= null &&!username.isEmpty()) {
```

权限管理员可以进行角色权限的管理



完整代码如下

User

```
class User {
    private int id;
    private String username;
    private String password;
    private Role role;

    public User(int id, String username, String password, Role role) {
        this.id = id;
        this.username = username;
        this.password = password;
        this.role = role;
    }

    public Role getRole() {
```

```
        return role;
    }
    public String getUsername(){
        return username;
    }

    public int getId() {
        return id;
    }

    public String getPassword() {
        return password;
    }
}
```

Role

```
Role
class Role {
    private int id;
    private String roleName;
    private Permission[] permissions;

    public Role(int id, String roleName, Permission[] permissions) {
        this.id = id;
        this.roleName = roleName;
        this.permissions = permissions;
    }

    public int getId() {
        return id;
    }

    public boolean hasPermission(Permission permission) {
        for (Permission p : permissions) {
            if (p.equals(permission)) {
                return true;
            }
        }
        return false;
    }

    public String getRoleName() {
        return roleName;
    }
}
```

Document

```
class Document {
    private int id;
    private String title;
    private String content;
```

```
    private int userId;

    public Document(int id, String title, String content, int userId) {
        this.id = id;
        this.title = title;
        this.content = content;
        this.userId = userId;
    }

    public int getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public String getContent() {
        return content;
    }

    public int getUserId() {
        return userId;
    }
}
```

Permission

```
public class Permission {
    private int id;
    private String permissionName;

    public Permission(int id, String permissionName) {
        this.id = id;
        this.permissionName = permissionName;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj instanceof Permission) {
            Permission other = (Permission) obj;
            return permissionName.equals(other.permissionName);
        }
        return false;
    }

    @Override
    public int hashCode() {
        return permissionName.hashCode();
    }

    public int getId() {
```

```
        return id;
    }

    public String getPermissionName() {
        return permissionName;
    }

    @Override
    public String toString() {
        return permissionName;
    }
}
```

UserInfoRetriever

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

class UserInfoRetriever {
    public static User getUser(String username) {
        Connection connection = DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return null;
        }
        try {
            String sql = "SELECT u.id, u.username, u.password, r.id AS role_id,
r.role_name " +
                    "FROM users u " +
                    "JOIN user_roles ur ON u.id = ur.user_id " +
                    "JOIN roles r ON ur.role_id = r.id " +
                    "WHERE u.username =?";
            PreparedStatement statement = connection.prepareStatement(sql);
            statement.setString(1, username);
            ResultSet resultSet = statement.executeQuery();
            if (resultSet.next()) {
                int userId = resultSet.getInt("id");
                String userPassword = resultSet.getString("password");
                int roleId = resultSet.getInt("role_id");
                String roleName = resultSet.getString("role_name");
                Role role = getRoleFromDatabase(roleId);
                return new User(userId, username, userPassword, role);
            }
            return null;
        } catch (SQLException e) {
            e.printStackTrace();
            return null;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (SQLException e) {
                e.printStackTrace();
```

```
            }
        }
    }

    public static Role getRoleFromDatabase(int roleId) {
        Connection connection = DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return null;
        }
        try {
            String sqlRole = "SELECT r.id, r.role_name, p.id AS permission_id,
p.permission_name " +
                    "FROM roles r " +
                    "JOIN role_permissions rp ON r.id = rp.role_id " +
                    "JOIN permissions p ON rp.permission_id = p.id " +
                    "WHERE r.id =?";
            PreparedStatement statementRole =
connection.prepareStatement(sqlRole);
            statementRole.setInt(1, roleId);
            ResultSet resultSetRole = statementRole.executeQuery();
            if (resultSetRole.next()) {
                String roleName = resultSetRole.getString("role_name");
                Permission[] permissions = new Permission[0];
                do {
                    int permissionId = resultSetRole.getInt("permission_id");
                    String permissionName =
resultSetRole.getString("permission_name");
                    Permission permission = new Permission(permissionId,
permissionName);
                    Permission[] newPermissions = new
Permission[permissions.length + 1];
                    System.arraycopy(permissions, 0, newPermissions, 0,
permissions.length);
                    newPermissions[permissions.length] = permission;
                    permissions = newPermissions;
                } while (resultSetRole.next());
                return new Role(roleId, roleName, permissions);
            }
            return null;
        } catch (SQLException e) {
            e.printStackTrace();
            return null;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

UserManager

```
class UserManager {
    public static boolean addUser(String username, String password, int roleId) {
        java.sql.Connection connection =
DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return false;
        }
        try {
            String sql = "INSERT INTO users (username, password) VALUES (?,?)";
            java.sql.PreparedStatement statement =
connection.prepareStatement(sql,
java.sql.PreparedStatement.RETURN_GENERATED_KEYS);
            statement.setString(1, username);
            statement.setString(2, password);
            int rowsAffected = statement.executeUpdate();
            if (rowsAffected > 0) {
                java.sql.ResultSet generatedKeys = statement.getGeneratedKeys();
                if (generatedKeys.next()) {
                    int userId = generatedKeys.getInt(1);
                    sql = "INSERT INTO user_roles (user_id, role_id) VALUES
(?,?)";
                    statement = connection.prepareStatement(sql);
                    statement.setInt(1, userId);
                    statement.setInt(2, roleId);
                    return statement.executeUpdate() > 0;
                }
            }
            return false;
        } catch (java.sql.SQLException e) {
            e.printStackTrace();
            return false;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (java.sql.SQLException e) {
                e.printStackTrace();
            }
        }
    }

    public static boolean deleteUser(int userId) {
        java.sql.Connection connection =
DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return false;
        }
        try {
            String sql = "DELETE FROM users WHERE id =?";
            java.sql.PreparedStatement statement =
connection.prepareStatement(sql);
            statement.setInt(1, userId);
```

```
            int rowsAffected = statement.executeUpdate();
            return rowsAffected > 0;
        } catch (java.sql.SQLException e) {
            e.printStackTrace();
            return false;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (java.sql.SQLException e) {
                e.printStackTrace();
            }
        }
    }

    public static boolean editUser(int userId, String username, String password,
int roleId) {
        java.sql.Connection connection =
DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return false;
        }
        try {
            String sql = "UPDATE users SET username =?, password =? WHERE id =?";
            java.sql.PreparedStatement statement =
connection.prepareStatement(sql);
            statement.setString(1, username);
            statement.setString(2, password);
            statement.setInt(3, userId);
            int rowsAffected = statement.executeUpdate();
            if (rowsAffected > 0) {
                sql = "UPDATE user_roles SET role_id =? WHERE user_id =?";
                statement = connection.prepareStatement(sql);
                statement.setInt(1, roleId);
                statement.setInt(2, userId);
                return statement.executeUpdate() > 0;
            }
            return false;
        } catch (java.sql.SQLException e) {
            e.printStackTrace();
            return false;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (java.sql.SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

PermissionChecker

```
class PermissionChecker {
    public static boolean checkPermission(User user, Permission permission) {
        return user.getRole().hasPermission(permission);

    }
}
```

PermissionManagement

```
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import javax.swing.*;
import javax.swing.border.TitledBorder;
import javax.swing.table.DefaultTableModel;

class PermissionManagement {
    public static void showPermissionMatrixDialog(Frame parentFrame) {
        JDialog permissionDialog = new JDialog(parentFrame, "Permission Matrix
Management", true);
        permissionDialog.setLayout(new BorderLayout());

        // 获取所有角色和权限信息
        List<Role> allRoles = getAllRoles();
        List<Permission> allPermissions = getAllPermissions();


        String[] columnNames = {"Permission Name"};
        for (Role role : allRoles) {
            columnNames = appendColumnName(columnNames, role.getRoleName());
        }


        Object[][] data = new Object[allPermissions.size()][columnNames.length];
        for (int i = 0; i < allPermissions.size(); i++) {
            Permission permission = allPermissions.get(i);
            data[i][0] = permission.getPermissionName();
            for (int j = 1; j < columnNames.length; j++) {
                Role role = allRoles.get(j - 1);
                boolean hasPermission = doesRoleHavePermission(role.getId(),
permission.getId());
                data[i][j] = hasPermission;
            }
        }
```

```java
        DefaultTableModel tableModel = new DefaultTableModel(data, columnNames) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return column > 0;
            }
        };


        JTable permissionTable = new JTable(tableModel);
        for (int i = 1; i < permissionTable.getColumnCount(); i++) {
            permissionTable.getColumnModel().getColumn(i).setCellRenderer(new
CheckBoxCellRenderer());
            permissionTable.getColumnModel().getColumn(i).setCellEditor(new
DefaultCellEditor(new JCheckBox()));
        }
        permissionDialog.add(new JScrollPane(permissionTable),
BorderLayout.CENTER);

        JButton saveButton = new JButton("Save Changes");
        saveButton.addActionListener(e -> {
            // 保存权限更改
            savePermissionMatrixChanges(allPermissions, allRoles, tableModel);
            permissionDialog.dispose();
            JOptionPane.showMessageDialog(parentFrame, "Permissions saved
successfully.");
        });

        permissionDialog.add(saveButton, BorderLayout.SOUTH);

        permissionDialog.pack();
        permissionDialog.setLocationRelativeTo(parentFrame);
        permissionDialog.setVisible(true);
    }

    // 获取所有权限
    private static List<Permission> getAllPermissions() {
        List<Permission> permissions = new ArrayList<>();
        Connection connection = DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return permissions;
        }
        try {
            String sql = "SELECT id, permission_name FROM permissions";
            PreparedStatement statement = connection.prepareStatement(sql);
            ResultSet resultSet = statement.executeQuery();
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String permissionName = resultSet.getString("permission_name");
                permissions.add(new Permission(id, permissionName));
            }
            return permissions;
        } catch (SQLException e) {
            e.printStackTrace();
            return permissions;
        } finally {
            try {
```

```
                if (connection!= null) {
                    connection.close();
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }

    // 获取所有角色
    private static List<Role> getAllRoles() {
        List<Role> roles = new ArrayList<>();
        Connection connection = DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return roles;
        }
        try {
            String sql = "SELECT id, role_name FROM roles";
            PreparedStatement statement = connection.prepareStatement(sql);
            ResultSet resultSet = statement.executeQuery();
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String roleName = resultSet.getString("role_name");
                roles.add(new Role(id, roleName, null));
            }
            return roles;
        } catch (SQLException e) {
            e.printStackTrace();
            return roles;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }

    // 判断角色法
    private static boolean doesRoleHavePermission(int roleId, int permissionId) {
        Connection connection = DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return false;
        }
        try {
            String sql = "SELECT * FROM role_permissions WHERE role_id =? AND
permission_id =?";
            PreparedStatement statement = connection.prepareStatement(sql);
            statement.setInt(1, roleId);
            statement.setInt(2, permissionId);
            ResultSet resultSet = statement.executeQuery();
            return resultSet.next();
        } catch (SQLException e) {
            e.printStackTrace();
```

```
            return false;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }

    // 保存权限矩阵
    private static void savePermissionMatrixChanges(List<Permission>
allPermissions, List<Role> allRoles, DefaultTableModel tableModel) {
        int rowCount = tableModel.getRowCount();
        int columnCount = tableModel.getColumnCount();
        for (int i = 0; i < rowCount; i++) {
            Permission permission = allPermissions.get(i);
            for (int j = 1; j < columnCount; j++) {
                Role role = allRoles.get(j - 1);
                boolean hasPermission = (boolean) tableModel.getValueAt(i, j);
                if (hasPermission!= doesRoleHavePermission(role.getId(),
permission.getId())) {
                    if (hasPermission) {
                        addPermissionToRole(role.getId(), permission.getId());
                    } else {
                        removePermissionFromRole(role.getId(),
permission.getId());
                    }
                }
            }
        }
    }

    // 将权限添加到角色
    private static void addPermissionToRole(int roleId, int permissionId) {
        Connection connection = DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return;
        }
        try {
            String sql = "INSERT INTO role_permissions (role_id, permission_id)
VALUES (?,?)";
            PreparedStatement statement = connection.prepareStatement(sql);
            statement.setInt(1, roleId);
            statement.setInt(2, permissionId);
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (SQLException e) {
```

```
                e.printStackTrace();
            }
        }
    }

    // 从角色中移除权限
    private static void removePermissionFromRole(int roleId, int permissionId) {
        Connection connection = DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return;
        }
        try {
            String sql = "DELETE FROM role_permissions WHERE role_id =? AND
permission_id =?";
            PreparedStatement statement = connection.prepareStatement(sql);
            statement.setInt(1, roleId);
            statement.setInt(2, permissionId);
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }

    private static String[] appendColumnName(String[] original, String newColumn)
{
        String[] newArray = new String[original.length + 1];
        System.arraycopy(original, 0, newArray, 0, original.length);
        newArray[original.length] = newColumn;
        return newArray;
    }

    static class CheckBoxCellRenderer extends JCheckBox implements
javax.swing.table.TableCellRenderer {
        public CheckBoxCellRenderer() {
            setHorizontalAlignment(SwingConstants.CENTER);
        }

        @Override
        public Component getTableCellRendererComponent(JTable table, Object
value, boolean isSelected, boolean hasFocus, int row, int column) {
            setSelected((boolean) value);
            setBorderPainted(false);
            setOpaque(false);
            return this;
        }
    }
}
```

DocumentManager

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
class DocumentManager {

    public static List<Document> getDocumentsForUser(User user) {
        List<Document> documents = new java.util.ArrayList<>();
        java.sql.Connection connection =
DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return documents;
        }
        try {
            String sql = "SELECT id, title, content, user_id FROM documents WHERE
user_id =?";
            java.sql.PreparedStatement statement =
connection.prepareStatement(sql);
            statement.setInt(1, user.getId());
            java.sql.ResultSet resultSet = statement.executeQuery();
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String title = resultSet.getString("title");
                String content = resultSet.getString("content");
                int userId = resultSet.getInt("user_id");
                documents.add(new Document(id, title, content, userId));
            }
            return documents;
        } catch (java.sql.SQLException e) {
            e.printStackTrace();
            return documents;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (java.sql.SQLException e) {
                e.printStackTrace();
            }
        }
    }

    public static boolean createDocument(User user, String title, String content)
{
        java.sql.Connection connection =
DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return false;
        }
        try {
```

```
            String sql = "INSERT INTO documents (title, content, user_id) VALUES
(?,?,?)";
            java.sql.PreparedStatement statement =
connection.prepareStatement(sql);
            statement.setString(1, title);
            statement.setString(2, content);
            statement.setInt(3, user.getId());
            int rowsAffected = statement.executeUpdate();
            return rowsAffected > 0;
        } catch (java.sql.SQLException e) {
            e.printStackTrace();
            return false;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (java.sql.SQLException e) {
                e.printStackTrace();
            }
        }
    }

    public static boolean deleteDocument(User user, int documentId) {
        if (!checkPermission(user, new Permission(0, "delete_document"))) {
            return false;
        }
        java.sql.Connection connection =
DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return false;
        }
        try {
            String sql = "DELETE FROM documents WHERE id =? AND user_id =?";
            java.sql.PreparedStatement statement =
connection.prepareStatement(sql);
            statement.setInt(1, documentId);
            statement.setInt(2, user.getId());
            int rowsAffected = statement.executeUpdate();
            return rowsAffected > 0;
        } catch (java.sql.SQLException e) {
            e.printStackTrace();
            return false;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (java.sql.SQLException e) {
                e.printStackTrace();
            }
        }
    }

    public static boolean editDocument(User user, int documentId, String title,
String content) {
```

```
        if (!checkPermission(user, new Permission(0, "edit_document"))) {
            return false;
        }
        java.sql.Connection connection =
DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return false;
        }
        try {
            String sql = "UPDATE documents SET title =?, content =? WHERE id =?
AND user_id =?";
            java.sql.PreparedStatement statement =
connection.prepareStatement(sql);
            statement.setString(1, title);
            statement.setString(2, content);
            statement.setInt(3, documentId);
            statement.setInt(4, user.getId());
            int rowsAffected = statement.executeUpdate();
            return rowsAffected > 0;
        } catch (java.sql.SQLException e) {
            e.printStackTrace();
            return false;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (java.sql.SQLException e) {
                e.printStackTrace();
            }
        }
    }

    private static boolean checkPermission(User user, Permission permission) {
        return PermissionChecker.checkPermission(user, permission);
    }
}
```

DatabaseConnectionManager

```
class DatabaseConnectionManager {
    private static final String url =
"jdbc:mysql://localhost:3306/notebook_rbac_database";
    private static final String username = "root";
    private static final String password = "12345678";

    public static java.sql.Connection getConnection() {
        try {
            return java.sql.DriverManager.getConnection(url, username, password);
        } catch (java.sql.SQLException e) {
            e.printStackTrace();
            return null;
        }
```

```
    }
}
```

LoginFrame

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

class LoginFrame extends JFrame {
    private JTextField usernameField;
    private JPasswordField passwordField;
    private JButton loginButton;

    public LoginFrame() {
        setTitle("Login");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JPanel panel = new JPanel(new GridLayout(3, 2));

        JLabel usernameLabel = new JLabel("Username:");
        usernameField = new JTextField();
        JLabel passwordLabel = new JLabel("Password:");
        passwordField = new JPasswordField();
        loginButton = new JButton("Login");

        panel.add(usernameLabel);
        panel.add(usernameField);
        panel.add(passwordLabel);
        panel.add(passwordField);
        panel.add(new JLabel());
        panel.add(loginButton);

        add(panel);

        pack();
        setLocationRelativeTo(null);

        loginButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                String username = usernameField.getText();
                char[] passwordChars = passwordField.getPassword();
                String password = new String(passwordChars);
                User user = UserInfoRetriever.getUser(username);
                if (user!= null && user.getPassword().equals(password)) {
                    dispose();
                    new NotebookAppWithRBAC(user);
                } else {
                    JOptionPane.showMessageDialog(LoginFrame.this, "Invalid
username or password.");
                }
            }
        });
```

```
    }
}
```

NotebookApp

```
import javax.swing.*;
import javax.swing.event.ListSelectionEvent;
import javax.swing.event.ListSelectionListener;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.List;

class NotebookApp {
    private JFrame frame;
    private JMenuBar menuBar;
    private JMenu fileMenu, userMenu, permissionMenu;
    private JMenuItem newDocumentItem, editDocumentItem, deleteDocumentItem,
viewDocumentsItem;
    private JMenuItem addUserItem, editUserItem, deleteUserItem;
    private JMenuItem managePermissionsItem;
    private JTextArea contentArea;
    private JTextField titleField;
    private JList<Document> documentList;
    private DefaultListModel<Document> documentListModel;
    private User currentUser;

    public NotebookApp(User user) {
        currentUser = user;
        frame = new JFrame("Notebook Application with RBAC");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // 创建菜单栏
        menuBar = new JMenuBar();
        fileMenu = new JMenu("File");
        userMenu = new JMenu("User Management");
        permissionMenu = new JMenu("Permission Management");
        newDocumentItem = new JMenuItem("New Document");
        editDocumentItem = new JMenuItem("Edit Document");
        deleteDocumentItem = new JMenuItem("Delete Document");
        viewDocumentsItem = new JMenuItem("View Documents");
        addUserItem = new JMenuItem("Add User");
        editUserItem = new JMenuItem("Edit User");
        deleteUserItem = new JMenuItem("Delete User");
        managePermissionsItem = new JMenuItem("Manage Permissions");
        fileMenu.add(newDocumentItem);
        fileMenu.add(editDocumentItem);
        fileMenu.add(deleteDocumentItem);
        fileMenu.add(viewDocumentsItem);
        userMenu.add(addUserItem);
        userMenu.add(editUserItem);
        userMenu.add(deleteUserItem);
        permissionMenu.add(managePermissionsItem);
        menuBar.add(fileMenu);
        menuBar.add(userMenu);
```

```
        menuBar.add(permissionMenu);

        JPanel contentPanel = new JPanel(new BorderLayout());
        titleField = new JTextField();
        contentArea = new JTextArea();
        contentPanel.add(titleField, BorderLayout.NORTH);
        contentPanel.add(new JScrollPane(contentArea), BorderLayout.CENTER);

        documentListModel = new DefaultListModel<>();
        documentList = new JList<>(documentListModel);
        documentList.addListSelectionListener(new ListSelectionListener() {
            @Override
            public void valueChanged(ListSelectionEvent e) {
                if (!e.getValueIsAdjusting()) {
                    Document selectedDocument = documentList.getSelectedValue();
                    if (selectedDocument!= null) {
                        titleField.setText(selectedDocument.getTitle());
                        contentArea.setText(selectedDocument.getContent());
                    }
                }
            }
        });

        JSplitPane splitPane = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT, new
JScrollPane(documentList), contentPanel);

        frame.setJMenuBar(menuBar);
        frame.add(splitPane);

        newDocumentItem.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (currentUser!= null) {
                    boolean hasPermission =
PermissionChecker.checkPermission(currentUser, new Permission(0,
"create_document"));
                    if (hasPermission) {
                        String title = JOptionPane.showInputDialog(frame, "Enter
document title:");
                        if (title!= null &&!title.isEmpty()) {
                            String content = JOptionPane.showInputDialog(frame,
"Enter document content:");
                            if (content!= null &&!content.isEmpty()) {
                                boolean success =
DocumentManager.createDocument(currentUser, title, content);
                                if (success) {
                                    updateDocumentList();
                                    JOptionPane.showMessageDialog(frame,
"Document created successfully.");
                                } else {
                                    JOptionPane.showMessageDialog(frame, "Failed
to create document.");
                                }
                            }
                        }
                    } else {
```

```
                        JOptionPane.showMessageDialog(frame, "You don't have
permission to create a document.");

                    }
                }
            }
        });

        editDocumentItem.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (currentUser!= null &&
PermissionChecker.checkPermission(currentUser, new Permission(0,
"edit_document"))) {
                    // 获取当前用户的文档列表
                    List<Document> userDocuments =
DocumentManager.getDocumentsForUser(currentUser);
                    // 创建一个临时的列表模型用于存储当前用户的文档信息字符串
                    DefaultListModel<String> tempListModel = new
DefaultListModel<>();
                    for (Document doc : userDocuments) {
                        tempListModel.addElement( doc.getTitle() + "; " +
doc.getContent());
                    }
                    // 创建一个临时的 JList 展示当前用户文档列表
                    JList<String> tempList = new JList<>(tempListModel);
                    int selectedIndex = JOptionPane.showOptionDialog(frame, new
JScrollPane(tempList), "Select a document to edit",
                            JOptionPane.OK_CANCEL_OPTION,
JOptionPane.PLAIN_MESSAGE, null, null, null);
                    if (selectedIndex!= -1) {
                        String selectedItem =
tempListModel.getElementAt(selectedIndex);
                        String[] parts = selectedItem.split("; ");
                        int documentId = Integer.parseInt(parts[0]);
                        Document selectedDocument = userDocuments.stream()
                                .filter(doc -> doc.getId() == documentId)
                                .findFirst()
                                .orElse(null);
                        if (selectedDocument!= null) {
                            String newTitle = JOptionPane.showInputDialog(frame,
"Enter new title:", selectedDocument.getTitle());
                            if (newTitle!= null &&!newTitle.isEmpty()) {
                                String newContent =
JOptionPane.showInputDialog(frame, "Enter new content:",
selectedDocument.getContent());
                                if (newContent!= null &&!newContent.isEmpty()) {
                                    boolean success =
DocumentManager.editDocument(currentUser, selectedDocument.getId(), newTitle,
newContent);
                                    if (success) {
                                        updateDocumentList();
                                        JOptionPane.showMessageDialog(frame,
"Document edited successfully.");
                                    } else {
```

```
                                        JOptionPane.showMessageDialog(frame,
"Failed to edit document.");
                                    }
                                }
                            }
                        } else {
                            JOptionPane.showMessageDialog(frame, "You don't have
permission to edit this document.");
                        }
                    }
                }
            }
        });

        deleteDocumentItem.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (currentUser!= null &&
PermissionChecker.checkPermission(currentUser, new Permission(0,
"delete_document"))) {
                    // 获取当前用户的文档列表
                    List<Document> userDocuments =
DocumentManager.getDocumentsForUser(currentUser);
                    // 创建一个临时的列表模型用于存储当前用户的文档信息字符串
                    DefaultListModel<String> tempListModel = new
DefaultListModel<>();
                    for (Document doc : userDocuments) {
                        tempListModel.addElement( doc.getTitle() + "; " +
doc.getContent());
                    }
                    // 创建一个临时的 JList 展示当前用户文档列表
                    JList<String> tempList = new JList<>(tempListModel);
                    int selectedIndex = JOptionPane.showOptionDialog(frame, new
JScrollPane(tempList), "Select a document to delete",
                            JOptionPane.OK_CANCEL_OPTION,
JOptionPane.PLAIN_MESSAGE, null, null, null);
                    if (selectedIndex!= -1) {
                        String selectedItem =
tempListModel.getElementAt(selectedIndex);
                        String[] parts = selectedItem.split("; ");
                        int documentId = Integer.parseInt(parts[0]);
                        Document selectedDocument = userDocuments.stream()
                                .filter(doc -> doc.getId() == documentId)
                                .findFirst()
                                .orElse(null);
                        if (selectedDocument!= null) {
                            int confirm = JOptionPane.showConfirmDialog(frame,
"Are you sure you want to delete this document?");
                            if (confirm == JOptionPane.YES_OPTION) {
                                boolean success =
DocumentManager.deleteDocument(currentUser, selectedDocument.getId());
                                if (success) {
                                    updateDocumentList();
                                    JOptionPane.showMessageDialog(frame,
"Document deleted successfully.");
                                } else {
```

```
                                    JOptionPane.showMessageDialog(frame, "Failed
to delete document.");
                                }
                            }
                        } else {
                            JOptionPane.showMessageDialog(frame, "You don't have
permission to delete this document.");
                        }
                    }
                }
            }
        });

        viewDocumentsItem.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (currentUser!= null) {
                    updateDocumentList();
                } else {
                    JOptionPane.showMessageDialog(frame, "Failed to retrieve user
information.");
                }
            }
        });

        addUserItem.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (currentUser!= null &&
PermissionChecker.checkPermission(currentUser, new Permission(0, "add_user"))) {
                    String username = JOptionPane.showInputDialog(frame, "Enter
new user's username:");
                    if (username!= null &&!username.isEmpty()) {
                        String password = JOptionPane.showInputDialog(frame,
"Enter new user's password:");
                        if (password!= null &&!password.isEmpty()) {
                            int roleChoice = JOptionPane.showOptionDialog(frame,
"Select user role:", "Choose Role",
                                    JOptionPane.DEFAULT_OPTION,
JOptionPane.QUESTION_MESSAGE, null,
                                    new String[]{"Normal User", "Admin",
"Permission Admin"}, "Normal User");
                            boolean success = UserManager.addUser(username,
password, roleChoice + 1);
                            if (success) {
                                JOptionPane.showMessageDialog(frame, "User added
successfully.");
                            } else {
                                JOptionPane.showMessageDialog(frame, "Failed to
add user.");
                            }
                        }
                    } else {
                        JOptionPane.showMessageDialog(frame, "You don't have
permission to add a user.");
                    }
```

```
                }else {
                    JOptionPane.showMessageDialog(frame, "You don't have
permission to manage permissions.");
                }
            }
        });

        editUserItem.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (currentUser!= null &&
PermissionChecker.checkPermission(currentUser, new Permission(0, "edit_user"))) {
                    int selectedUserId =
Integer.parseInt(JOptionPane.showInputDialog(frame, "Enter the ID of the user to
edit:"));
                    if (selectedUserId > 0) {
                        User userToEdit = getUserById(selectedUserId);
                        if (userToEdit!= null) {
                            String newUsername =
JOptionPane.showInputDialog(frame, "Enter new username:",
userToEdit.getUsername());
                            if (newUsername!= null &&!newUsername.isEmpty()) {
                                String newPassword =
JOptionPane.showInputDialog(frame, "Enter new password:",
userToEdit.getPassword());
                                if (newPassword!= null &&!newPassword.isEmpty())
{
                                    int roleChoice =
JOptionPane.showOptionDialog(frame, "Select new user role:", "Choose Role",
                                            JOptionPane.DEFAULT_OPTION,
JOptionPane.QUESTION_MESSAGE, null,
                                            new String[]{"Normal User", "Admin",
"Permission Admin"}, userToEdit.getRole().getRoleName());
                                    boolean success =
UserManager.editUser(selectedUserId, newUsername, newPassword, roleChoice + 1);
                                    if (success) {
                                        JOptionPane.showMessageDialog(frame,
"User edited successfully.");
                                    } else {
                                        JOptionPane.showMessageDialog(frame,
"Failed to edit user.");
                                    }
                                }
                            }
                        } else {
                            JOptionPane.showMessageDialog(frame, "User not
found.");
                        }
                    } else {
                        JOptionPane.showMessageDialog(frame, "You don't have
permission to edit this user.");
                    }
                }else {
                    JOptionPane.showMessageDialog(frame, "You don't have
permission to manage permissions.");
                }
```

```
            }
        });

        deleteUserItem.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (currentUser!= null &&
PermissionChecker.checkPermission(currentUser, new Permission(0, "delete_user")))
{
                    int selectedUserId =
Integer.parseInt(JOptionPane.showInputDialog(frame, "Enter the ID of the user to
delete:"));
                    if (selectedUserId > 0) {
                        int confirm = JOptionPane.showConfirmDialog(frame, "Are
you sure you want to delete this user?");
                        if (confirm == JOptionPane.YES_OPTION) {
                            boolean success =
UserManager.deleteUser(selectedUserId);
                            if (success) {
                                JOptionPane.showMessageDialog(frame, "User
deleted successfully.");
                            } else {
                                JOptionPane.showMessageDialog(frame, "Failed to
delete user.");
                            }
                        }
                    } else {
                        JOptionPane.showMessageDialog(frame, "You don't have
permission to delete this user.");
                    }
                }else {
                    JOptionPane.showMessageDialog(frame, "You don't have
permission to manage permissions.");
                }
            }
        });

        managePermissionsItem.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (currentUser!= null &&
PermissionChecker.checkPermission(currentUser, new Permission(0,
"manage_permissions"))) {
                    PermissionManagement.showPermissionMatrixDialog(frame);
                } else {
                    JOptionPane.showMessageDialog(frame, "You don't have
permission to manage permissions.");
                }
            }
        });

        frame.pack();
        frame.setVisible(true);

        updateDocumentList();
    }
```

```java
    private void updateDocumentList() {
        documentListModel.clear();
        List<Document> documents =
DocumentManager.getDocumentsForUser(currentUser);
        for (Document document : documents) {
            documentListModel.addElement(document);
        }
    }

    private User getUserById(int userId) {
        java.sql.Connection connection =
DatabaseConnectionManager.getConnection();
        if (connection == null) {
            return null;
        }
        try {
            String sql = "SELECT u.id, u.username, u.password, r.id AS role_id,
r.role_name " +
                    "FROM users u " +
                    "JOIN user_roles ur ON u.id = ur.user_id " +
                    "JOIN roles r ON ur.role_id = r.id " +
                    "WHERE u.id =?";
            java.sql.PreparedStatement statement =
connection.prepareStatement(sql);
            statement.setInt(1, userId);
            java.sql.ResultSet resultSet = statement.executeQuery();
            if (resultSet.next()) {
                int id = resultSet.getInt("id");
                String username = resultSet.getString("username");
                String password = resultSet.getString("password");
                int roleId = resultSet.getInt("role_id");
                String roleName = resultSet.getString("role_name");
                Role role = UserInfoRetriever.getRoleFromDatabase(roleId);
                return new User(id, username, password, role);
            }
            return null;
        } catch (java.sql.SQLException e) {
            e.printStackTrace();
            return null;
        } finally {
            try {
                if (connection!= null) {
                    connection.close();
                }
            } catch (java.sql.SQLException e) {
                e.printStackTrace();
            }
        }
    }

    public static void main(String[] args) {
        new LoginFrame().setVisible(true);
    }
}
```